



System design

Video Conferencing (e.g. Zoom) ★ Premium



Hard



45 mins



32 done

Report an issue 🛠️

Question

Design a browser-based video conferencing application similar to Zoom, Google Meet, or Whereby that lets authenticated users preview their camera and microphone before joining, participate in a multi-person meeting, mute and unmute themselves, toggle their camera, switch between grid and speaker layouts, share their screen, and send basic in-meeting chat messages.

Focus on the front-end architecture and the client/server contract for joining a room, establishing and maintaining media connections, handling real-time participant state, and adapting gracefully to changing network quality. You do not need to go deep into scheduling, recording, breakout rooms, captions, webinars, native applications, or server-side media transcoding details.

Assume one active screen share at a time. Chat is session-scoped: acknowledged messages can be replayed after a reconnect, but long-term chat retention after the meeting is out of scope.

Real-life examples

These products demonstrate the prejoin, in-meeting media, layout, and control patterns in scope.

- <https://zoom.us>
- <https://meet.google.com>
- <https://teams.microsoft.com>

- <https://whereby.com>

Requirements

Use the following baseline unless the interviewer changes the product boundary.

- Support a desktop-first web meeting room for up to roughly 25 active participants.
 - Include a prejoin flow for camera and microphone permissions, device selection, and local preview.
 - Support participant roster updates, active speaker indication, mute/camera controls, one active screen share, and session-scoped in-meeting chat.
 - Handle network degradation, reconnects, and quality adaptation gracefully.
 - You do not need to cover scheduling, recording, breakout rooms, captions, webinars, or native applications.
-

Requirements exploration

Designing a browser-based video conferencing application is a challenging frontend system design exercise because it combines real-time media, device permissions, networking tradeoffs, and rapidly changing UI state in one product surface.

Unlike simpler media products, a conferencing app has to establish live connections, recover from flaky networks, adapt audio and video quality in real time, and keep the meeting UI responsive while participants join, leave, mute, and share screens.

What are the core features to be supported?

The baseline covers one interactive meeting rather than the surrounding scheduling product.

- Prejoin device check with local camera and microphone preview.
- Join an existing meeting room.
- See remote participants in a grid or speaker-focused layout.
- Mute and unmute microphone, turn camera on and off, and switch devices.

- Share the screen during the meeting.
- Send and receive basic chat messages.

How many participants do we need to support?

Assume up to roughly 25 active participants in a meeting. That is large enough that a naive peer-to-peer mesh becomes impractical, but still small enough that we can focus on an interactive meeting room rather than a webinar product.

What devices and platforms are in scope?

Desktop web is the primary target. The layout can adapt to tablet and mobile browsers where possible, but native desktop applications are out of scope.

What are the non-functional requirements?

Prioritize conversational continuity and understandable recovery over maximum video quality.

- Fast join time after the user clicks Join.
- Low enough latency for natural conversation quality.
- Audio should remain stable even when network conditions degrade.
- Video quality should adapt dynamically to bandwidth and device constraints.
- The app should recover automatically from brief disconnects.
- The meeting UI should stay responsive even while participant state changes frequently.
- Keyboard and screen reader support should be considered for core meeting actions.

What is explicitly out of scope?

Scheduling, recording, webinars, captions, breakout rooms, reactions, end-to-end encryption, virtual backgrounds, and desktop applications. We can mention some of them later as advanced topics, but they are not part of the main design.

Exercise Beta

A team is defining a Zoom-like browser meeting client before selecting media topology and protocols. Which scope gives that decision a useful foundation?

- 1 Define codecs, TURN deployment, and signaling messages in detail, using current Zoom behavior as the implicit product contract.
- 2 Scope a one-to-one call first, assume peer-to-peer WebRTC, and leave group-size and recovery constraints to implementation.
- 3 Treat scheduling, live media, recording, webinars, captions, breakout rooms, reactions, and virtual backgrounds as equal baseline concerns.
- 4 Bound participant scale, active media, target browsers, essential meeting actions, quality priorities, reconnect behavior, accessibility, and excluded follow-ups.



Reveal answer

Check answer ↩

The design below approaches WebRTC from a frontend architecture point of view. The goal is to establish clear ownership and lifecycle boundaries, not to reproduce every detail of the networking stack.

Background

Before WebRTC, browser-based video conferencing was much less standardized. Products often depended on plugins such as Flash, Java applets, or custom native applications because browsers did not expose a common way to access cameras, microphones, or low-latency real-time media transport directly from the page.

WebRTC made plugin-free conferencing practical by giving browsers a standard set of APIs for media capture and real-time communication. That did not remove the need for application infrastructure, though: products still need signaling to coordinate session setup, and they usually rely on SFUs plus STUN/TURN services to make multi-party meetings reliable at scale.

This question uses a few networking and media terms that many frontend engineers do not work with every day. Here's a short primer to help frame the architectural choices without being a full WebRTC tutorial.

Glossary

Core browser APIs & concepts

- **WebRTC:** A browser-standard set of APIs and protocols for capturing media and transporting real-time audio, video, and optional data between endpoints.
- **Signaling:** Messages used to help endpoints establish and maintain a media session. In this design, signaling carries offers, answers, ICE candidates, participant presence changes, chat messages, and other meeting control events.
- **SDP (Session Description Protocol):** The text-based format used inside WebRTC offers and answers to describe media details such as which tracks, codecs, and connection parameters each side supports.
- **RTCPeerConnection** : The browser primitive that manages a WebRTC connection. It handles negotiation, ICE, track publishing and receiving, and connection state changes.

Connectivity and NAT traversal

- **NAT (Network Address Translation):** The router behavior that lets many devices in a home or office share one public IP address. This improves address reuse, but it also makes direct inbound connectivity between two browsers harder.
- **ICE (Interactive Connectivity Establishment):** The process of finding a working network path between endpoints, often by trying several candidates before settling on one.
- **STUN (Session Traversal Utilities for NAT):** A helper service that lets a client discover its public-facing network address so it can attempt direct connectivity.
- **TURN (Traversal Using Relays around NAT):** A relay service used when direct connectivity is not possible. It is more expensive than direct transport, but critical for reliability.

WebRTC network topologies

- **Mesh:** Each participant sends media directly to every other participant. This works for very small rooms (2-4 people), but upload and download costs grow too quickly

as the room size increases.

- **SFU (Selective Forwarding Unit):** A server receives multiple media streams and forwards selected streams to each participant without mixing them together. This is the standard baseline for modern multi-party conferencing on the web.
- **MCU (Multipoint Control Unit):** A server mixes participant streams before sending them back to clients. This simplifies the client but increases server cost and reduces layout flexibility.

If you want a deeper primer after this section, the official browser documentation linked in the references section is the best next step.

We will use the terms above throughout the rest of the article when discussing join flow, media transport, and quality adaptation.

Architecture / high-level design

There are three important architectural decisions to make here: the rendering approach, the WebRTC network topology, and the communication model.

Rendering approach

This product is highly interactive, requires JavaScript for media and device APIs, and is generally used by authenticated users. CSR is the right default because:

- The prejoin lobby, meeting shell, chat panel, and participant layout all depend heavily on client-side state.
- Media connections, permission prompts, and reconnect logic only exist on the client.
- SEO is not required for the meeting room itself.

The server can still return a minimal HTML shell and bootstrapping JSON for faster startup, but an SSR-heavy architecture does not meaningfully reduce the complexity of the conferencing experience.

WebRTC network topology

For a room of roughly 25 active participants, a direct mesh topology (with the client uploading a separate media stream to every other participant) becomes too expensive in bandwidth and decode work, while Multipoint Control Unit (MCU)-style server mixing reduces layout flexibility and pushes more cost to the server.

Selective Forwarding Unit (SFU) is the best default here because each participant maintains one upstream peer connection carrying a bounded set of audio, camera, and optional screen-share tracks and encodings. The server forwards selected encodings to receivers, while the client keeps control over layout, active-speaker behavior, and per-tile quality decisions. Upload no longer grows with the number of receivers, although it still depends on the local publications, simulcast layers, bitrates, and codecs.

✓ Default to SFU for rooms above a handful of participants

Mesh is fine for 2-4 person calls but collapses at ~25 participants because every client pays upload and decode costs for every peer. MCU centralizes mixing at the cost of layout flexibility and server spend. An SFU bounds each publisher's transport topology while leaving layout, active-speaker, and per-tile quality decisions on the client.

Communication model

There are many different kinds of data involved in a video conferencing app, and the transport mode used for each kind of data differs according to its properties.

- The **bootstrap API** uses HTTP to return meeting metadata, capability flags, ICE server configuration, and the information needed to open the session.
- The **signaling service** uses WebSocket to carry participant join/leave events, active speaker updates, chat messages, meeting control state, and SDP / ICE negotiation messages.
- The **media path** uses WebRTC to publish local tracks and receive remote tracks through the SFU.

This split keeps product state and control flow on a simpler real-time channel (WebSockets) while reserving WebRTC for the low-latency media path.

WebRTC encrypts each browser-to-SFU hop with DTLS-SRTP, but the SFU can access media while forwarding it and is therefore inside the trust boundary. Application-level media encryption that hides content from the SFU is a follow-up.

Therefore, the best architectural choices would be a CSR-first single-page application backed by a bootstrap API, a WebSocket signaling service, and WebRTC over an SFU. For a video conferencing application, the interesting front-end problems are not SEO or route rendering; they are managing device permissions, the `RTCPeerConnection` lifecycle, real-time participant state, adaptive subscriptions, and stable media elements in the meeting UI.

Component responsibilities

The browser components and session infrastructure divide ownership as follows.

Session infrastructure

The backend services establish the session, synchronize room state, and route media.

- **Bootstrap API:** Authenticates the user, authorizes meeting access, and returns initial meeting data plus ICE server configuration.
- **Signaling service:** Coordinates room presence, chat, active speaker updates, and the offer/answer plus ICE candidate exchange required to establish media sessions.
- **SFU:** Receives published audio/video tracks and forwards the right streams to each participant based on the room state.
- **STUN/TURN services:** Help ICE establish a working path and provide relay candidates when direct connectivity is not possible.

Client application

The browser keeps product state separate from the live media objects owned by WebRTC.

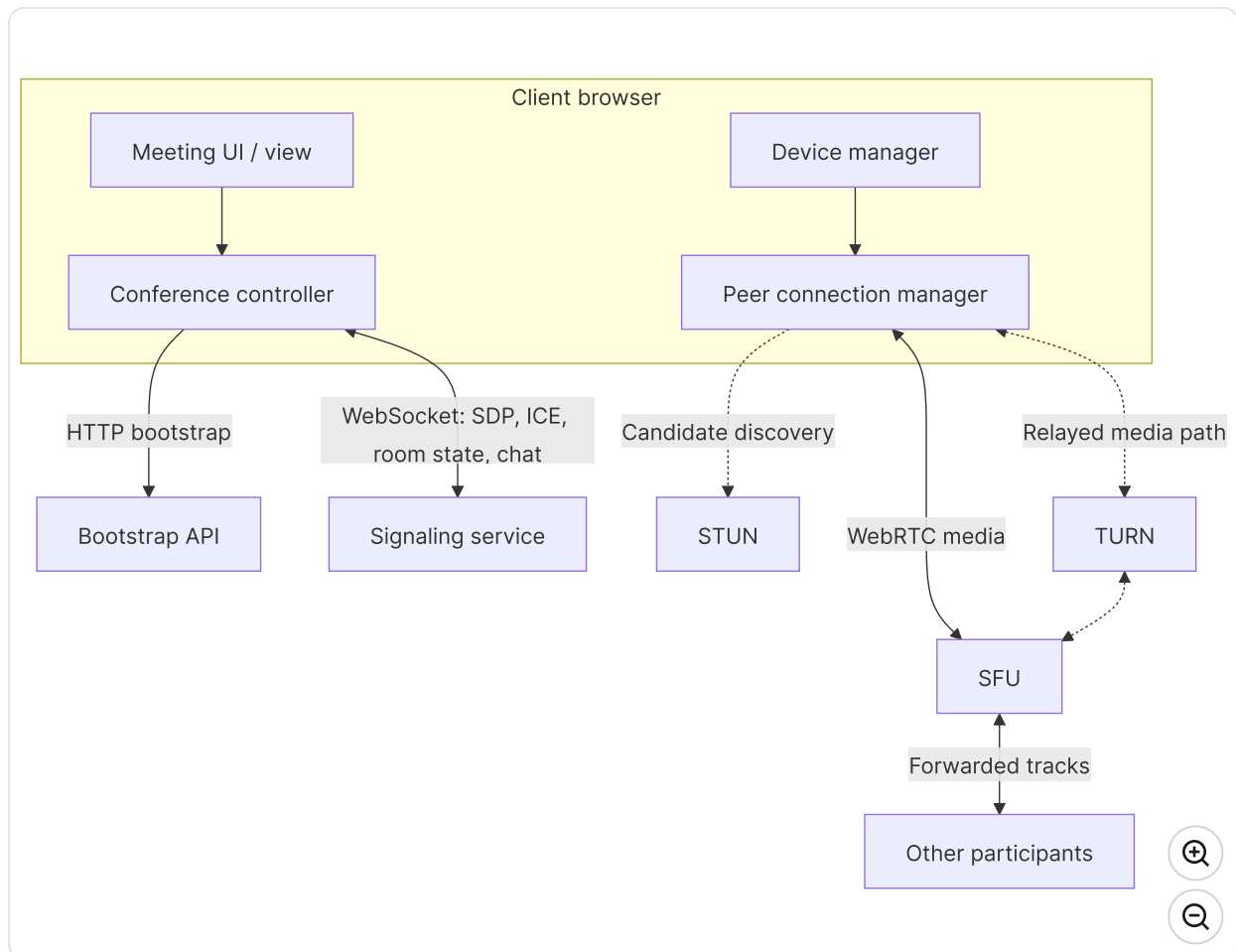
- **App shell:** Owns routing, top-level layout, and meeting-wide state that must stay mounted across room interactions.
- **Prejoin lobby:** Requests permissions, renders the local preview, allows device selection, and surfaces readiness or failure states before join.

- **Conference controller:** Owns session lifecycle, signaling connection, reconnect logic, and the mapping between user actions and meeting state updates.
- **Peer connection manager:** Wraps `RTCPeerConnection`, attaches or replaces local tracks, listens for remote tracks, and reacts to negotiation or connection-state changes.
- **Device manager:** Tracks camera and microphone devices, optional speaker-output selection where supported, and current permission plus selection state.
- **Layout manager:** Decides which participants are visible, who is dominant, and which tiles deserve higher-quality subscriptions.
- **Chat and presence panel:** Renders participant roster state, chat history, and transient room activity.

Why keep chat and controls on WebSocket?

In this scoped design, chat, presence, and meeting control updates are simpler over the signaling channel than over `RTCDataChannel`. They need predictable ordering, straightforward reconnect behavior, and easy server fan-out. Media still uses WebRTC, but product state stays on WebSocket, where it is easier to reason about and recover.

This division of responsibilities is similar in spirit to the Chat App system design, where real-time events and reconnect behavior are important, but the media path here is much more specialized.



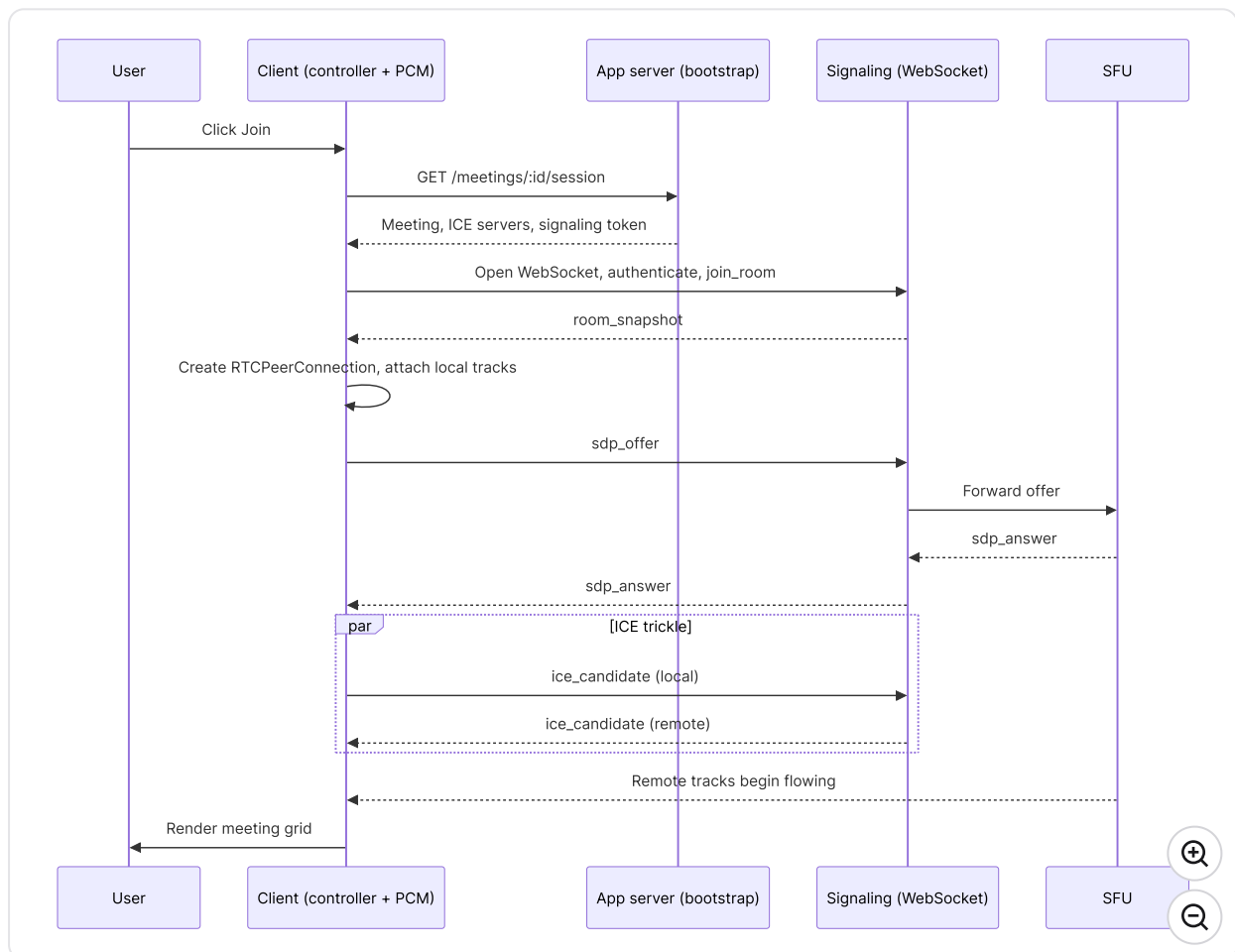
Client, signaling, SFU, and TURN at a glance

Call setup flow

The setup flow uses HTTP once for bootstrap, WebSocket for signaling, and WebRTC for the selected browser-to-SFU media path.

1. The client fetches meeting metadata, capability flags, and ICE server configuration from the application server over HTTP.
2. In the prejoin lobby, the browser requests camera and microphone access with `getUserMedia()` and renders a local preview.
3. When the user joins, the client opens a WebSocket signaling channel and creates an `RTCPeerConnection`.
4. The browser sends SDP offers and the SFU returns SDP answers over the signaling channel.
5. The browser first attempts direct paths to the SFU using ICE and STUN-discovered candidates.
6. If no direct path works, ICE can select TURN relay candidates from the configured servers.

7. Once connected, local audio, video, and optional screen-share tracks flow through WebRTC to the SFU, which forwards the right remote tracks back to each participant.
8. The peer connection manager publishes local tracks, receives remote tracks, and reacts to negotiation or ICE state changes.
9. The layout manager decides which participants should be emphasized and which subscriptions deserve higher quality.
10. The chat and presence panel reacts to signaling updates for roster changes, active speaker state, and messages.
11. The conference controller coordinates reconnect, recovery, and resynchronization when the network or room state changes.



Join meeting with SDP offer/answer and ICE exchange

The browser is the only SDP offerer. Initial setup, publication changes, and ICE restart enter one serialized queue that waits for a stable signaling state and tags each offer and answer with a `negotiationId`; stale answers are ignored. Perfect negotiation is a follow-up if the product later requires both sides to initiate offers.

Exercise 1 of 3 Beta

A signed-in browser meeting has a prejoin device check, live media, participant churn, and reconnect behavior. Which rendering model gives those lifecycles the clearest ownership?

- 1 Use a multi-page flow that reloads whenever a participant joins, leaves, or starts sharing.
- 2 Return a lightweight shell and bootstrap data, then let a client-rendered application own the prejoin and active meeting session.
- 3 Render a static meeting document and use only native media controls for participant updates.
- 4 Server-render the participant grid after every room event so the browser mainly attaches click handlers.



Reveal answer

Check answer ↩

Data model

Most of the interesting data in a conferencing app is session-scoped and changes quickly while the meeting is live. Unlike a chat app, we do not need a durable client-side database as the baseline. The most important modeling decision is separating synchronized meeting state from browser-owned media objects. The bootstrap HTTP response can stay relatively small and setup-focused, while the authoritative room state arrives over signaling after the user joins.

Entity	Source	Belongs to	Fields
Meeting	Server	App shell / conference controller	id , title , selfParticipantId , capabilities , iceServers , signaling

Entity	Source	Belongs to	Fields
Participant	Server via WebSocket	Roster, layout manager, chat panel	id , displayName , role , presence , micIntent , cameraIntent , connectionQuality
TrackPublication	Server via WebSocket	Peer connection manager, layout manager	publicationId , participantId , kind , source , mid , simulcastLayers
Subscription	Client, acknowledged by server	Layout manager, peer connection manager	publicationId , desiredLayer , appliedLayer , priority , pendingCommandId
ChatMessage	Server via WebSocket	Chat and presence panel	messageId , clientId , participantId , roomSeq , text , sentAt
ScreenShareSession	Server via WebSocket	Layout manager, participant tiles	participantId , publicationId , startedAt

Entity	Source	Belongs to	Fields
LocalMedia	Client	Prejoin lobby, conference controller	selectedMicId , selectedCameraId , selectedSpeakerId , desiredMicEnabled , desiredCameraEnabled , actualMicState , actualCameraState
Permissions	Client	Prejoin lobby, device manager	camera , microphone , captureAttemptId
ScreenShareState	Client	Conference controller, layout manager	status , trackId , surfaceType , withAudio , lastError
ConnectionState	Client	Conference controller	meetingStatus , signalingStatus , iceStatus , reconnectAttemptCount , lastError
LayoutState	Client	Layout manager	mode , pinnedParticipantId , dominantSpeakerId , visibleParticipantIds
StatsSnapshot	Client (derived from periodic getStats() polling)	Conference controller, layout manager	publicationId , direction , packetLoss , rtt , jitter , bitrate , decodePressure , updatedAt

These entity shapes do not need to match the wire payloads one-to-one. For example, the bootstrap response can be normalized so that `selfParticipant` becomes a `Participant` plus `Meeting.selfParticipantId`, while signaling connection details are folded into meeting or session state.

Track publications are the canonical source for actual media availability; participant media badges are derived from them, while acknowledged mic and camera intent remains participant metadata. Receiver-specific subscription choices stay in the local `Subscription` records rather than on shared publications.

Server-originated meeting state

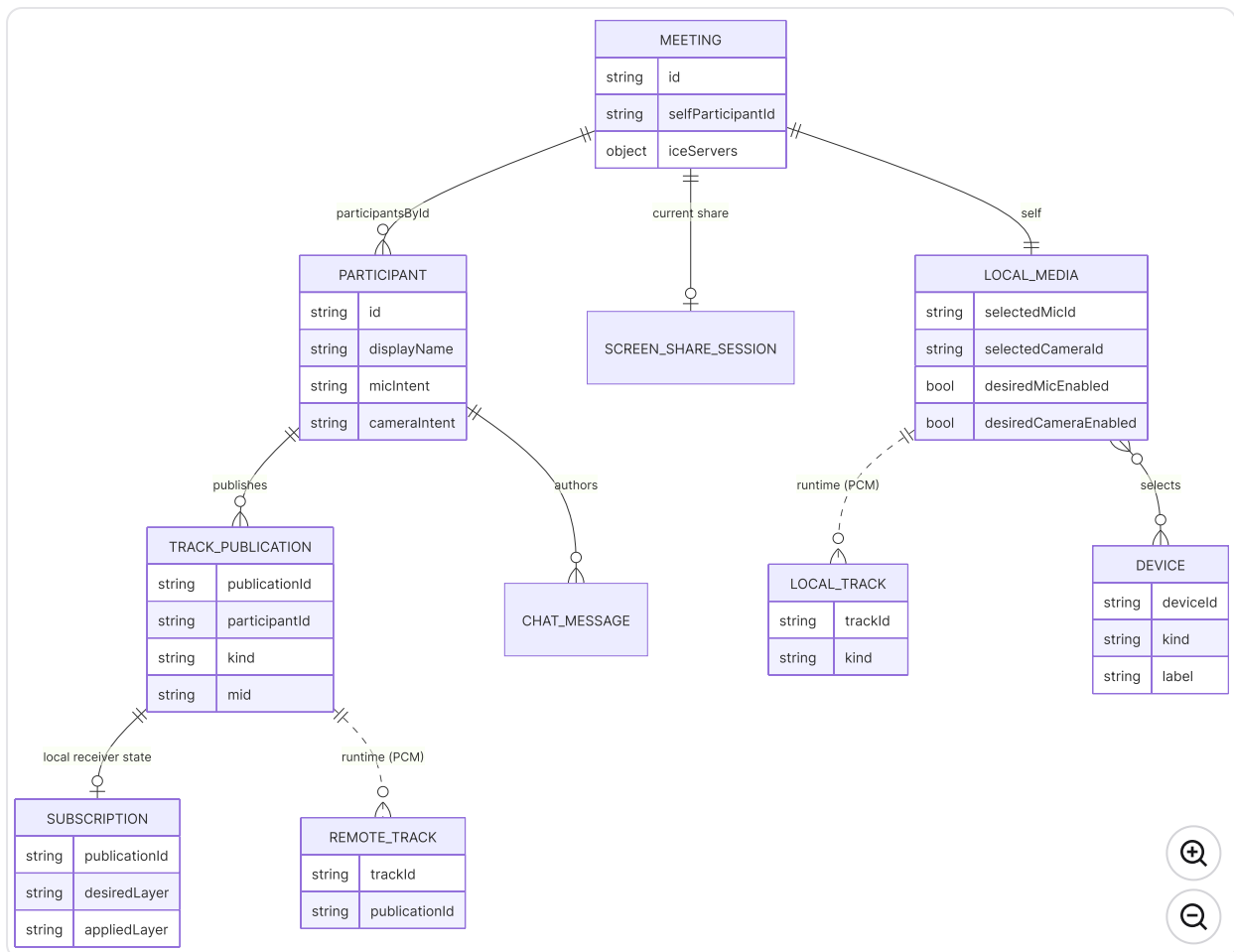
The synchronized meeting state sent over WebSocket after join should live in a normalized client store because the same participant and track data are rendered in multiple places at once:

- Roster and participant tiles both need participant identity and mute/camera state.
- Active speaker and pinned layout both need track and participant metadata.
- Chat messages reference the same participants already shown elsewhere in the meeting UI.

A normalized shape keeps those updates consistent:

- `participantsById`
- `trackPublicationsById`
- `chatMessagesById`
- `screenShareSession`
- ordered lists such as `participantOrder` or `chatMessageIds`

This is a better fit than storing one large server payload per view because participant state changes arrive continuously and need to update multiple UI surfaces at once.



Normalized conferencing store with runtime media held outside

Client-owned meeting state

Some state belongs entirely to the browser tab and should not come from the server:

- **LocalMedia** tracks current microphone and camera selection, whether the local user intends to publish those tracks, and the preferred audio output device where browser support exists.
- **Permissions** reflects whether the browser has granted or denied access to camera and microphone.
- **ScreenShareState** tracks the transient state of display capture. Unlike camera and microphone access, screen-share permission cannot be silently reused and must be requested via a user gesture each time.
- **ConnectionState** tracks whether the room is joining, connected, reconnecting, or failed, along with signaling and ICE health.
- **LayoutState** controls local presentation choices such as grid vs. speaker view and who is pinned.

- `StatsSnapshot` is a reduced summary derived from WebRTC stats APIs for UI decisions and quality indicators.

These fields are ideal for an in-memory store because they change frequently and are tightly coupled to the active tab.

Keep live media objects outside the serializable store

`MediaStream`, `MediaStreamTrack`, and `RTCPeerConnection` should not be stored in the serializable app state. They are browser-managed objects, not plain data:

★ Do not put `MediaStream` or `RTCPeerConnection` in your app store

These are live browser objects that mutate in response to network and hardware events, not plain data. Putting them in a serializable store causes unnecessary rerenders, subtle lifecycle bugs, and makes reconnect reasoning harder. Keep stable IDs and status flags in the store, and let the peer connection manager own the runtime objects imperatively.

- They are not safely serializable.
- They change in response to browser and network events at a much higher frequency than normal UI state.
- They often need imperative lifecycle handling, such as attaching tracks to media elements, swapping tracks, or reacting to connection callbacks.

Instead, the app state should store stable identifiers and status flags, while the peer connection manager owns the live objects in memory:

- Treat `publicationId` as the stable application identity. The SFU returns the negotiated `{ publicationId, mid }` binding in its answer; when `track` fires, the peer connection manager resolves `event.transceiver.mid` to the publication instead of assuming the sender's `MediaStreamTrack.id` survives the SFU hop.
- Keep media element refs and track attachment logic close to the rendering layer.
- Derive lightweight UI state from those objects rather than exposing the raw objects throughout the app.

This is similar to how the Chat App system design separates durable synced data from transient controller state, but conferencing has a stricter boundary because the media

objects are much more specialized.

What should be persisted locally?

Only lightweight preferences are worth persisting across sessions:

- Last-used microphone and camera device IDs, plus preferred speaker output where browser support exists.
- Preferred layout mode, if the product wants to remember it.
- Optional defaults such as joining with microphone muted or camera off.

Everything else should stay in memory:

- Live participant presence
- Active speaker state
- Connection health
- Stats snapshots
- Current screen-share session
- WebRTC runtime objects

If the user refreshes the page, the client should rebuild this state from the bootstrap response, signaling resynchronization, and fresh media setup rather than try to restore a stale local copy.

Exercise 1 of 3 Beta

A participant can publish several tracks, WebRTC stats arrive every second, and the UI must recover from a signaling snapshot. Which modeling choices keep state serializable without making rendering own media lifecycles? Select all that apply.

1 Normalize participants, publications, and chat messages by stable IDs, and derive participant media badges from publication state. ☐

2 Let a peer connection manager own RTCPeerConnection, MediaStream, and MediaStreamTrack objects outside the serializable store. ☐

3

Render a cached participant roster immediately, then merge the signaling snapshot into it by participant ID.

☐

4

Normalize every raw stats report by participant ID in the global store so any quality component can derive its own metric.

☐

5

Keep permissions, desired and actual local media state, receiver subscriptions, layout, connection status, and reduced quality summaries as client-owned state.

☐

Reveal answer

Check answer ↩

Interface definition (API)

The interface surface is intentionally small: one bootstrap request before join, one long-lived WebSocket for real-time non-media state, and one WebRTC session for media tracks.

Protocol matrix

Each interface has one responsibility and a different lifecycle.

Source	Destination	Protocol	Purpose
Browser	Application server	HTTP	Fetch meeting bootstrap data before join
Browser	Signaling service	WebSocket	Join room, sync participant state, send chat messages, exchange SDP / ICE messages, resynchronize after reconnect
Browser	SFU	WebRTC	Publish local media and receive subscribed remote media

Keeping these interfaces separate makes the client easier to reason about:

- HTTP is good for the initial authenticated fetch.
- WebSocket is good for ordered real-time room state and control flow.
- WebRTC is good for low-latency audio and video transport.

Bootstrap API

Before the client opens signaling or creates media sessions, it should fetch a single bootstrap payload with `Cache-Control: no-store`. This payload is for session setup and prejoin rendering; the authoritative live room state should come from signaling after the user joins. Signaling and TURN credentials are short-lived, scoped to the meeting and participant, and redacted from normal logs.

Field	Value
HTTP Method	GET
Path	/api/meetings/:meetingId/session
Description	Returns everything the browser needs to render the prejoin lobby and establish the session

Example response:

```
1  {
2    "meeting": {
3      "id": "meeting_123",
4      "title": "Weekly Product Sync",
5      "capabilities": {
6        "chat": true,
7        "screenShare": true
8      }
9    },
10   "selfParticipant": {
11     "id": "participant_self",
12     "displayName": "Alice"
```

```
13   },
14   "iceServers": [
15     { "urls": ["stun:stun.example.com:3478"] },
16     {
17       "urls": ["turn:turn.example.com:3478"],
18       "username": "user",
19       "credential": "secret",
20       "expiresAt": "2026-08-22T10:05:00Z"
21     }
22   ],
23   "signaling": {
24     "url": "wss://signal.example.com/meetings/meeting_123",
25     "token": "session_token",
26     "expiresAt": "2026-08-22T10:05:00Z"
27   }
28 }
```

This response should be enough to:

- Render the meeting title and local participant identity in prejoin.
- Initialize the prejoin lobby.
- Create `RTCPeerConnection` with the correct ICE server configuration.
- Open the authenticated WebSocket session when the user joins.

The full synchronized roster, track publications, and other live room state should come from the first `room_snapshot` after the signaling connection is established.

Signaling channel

The WebSocket should carry small, typed messages instead of many bespoke endpoints. Open it over WSS without putting the token in the URL, then send the bootstrap token as the first application message. The server validates `Origin`, binds the authenticated socket to one meeting and participant, rejects pre-authentication traffic, validates message schemas and sizes, and authorizes every command. A simple event is enough:

```
1  {
2    "type": "participant_joined",
3    "payload": {
```

```
4     "id": "participant_3",  
5     "displayName": "Carol"  
6   }  
7 }
```

The most important event types are:

- Room synchronization:
 - `room_snapshot` : full meeting snapshot on first join or after reconnect
 - `participant_joined`
 - `participant_left`
 - `participant_updated`
 - `meeting_ended`
- Chat and presence:
 - `chat_message`
 - `active_speaker_changed`
- Media metadata:
 - `track_published`
 - `track_unpublished`
- Negotiation:
 - `sdp_answer`
 - `ice_candidate`

The same channel can also carry client-originated commands such as:

- `join_room`
- `leave_room`
- `update_participant_state`
- `send_chat_message`
- `update_subscription_preferences`
- `sdp_offer`

- `ice_candidate`

Every `room_snapshot` includes a room `generation` and the last `eventSeq`; every later room event carries the same generation and a monotonically increasing sequence. While fetching a reconnect snapshot, buffer newer events, install the snapshot atomically, then drain only contiguous events. A gap or generation mismatch triggers another snapshot rather than applying uncertain state.

Chat is scoped to the meeting session and includes `clientId`, `serverMessageId`, `roomId`, `roomSeq`, sender, timestamp, and an acknowledgment or terminal error. Retrying the same client ID is idempotent, and a bounded recent history in `room_snapshot` fills reconnect gaps; messages are length- and rate-limited and rendered as text rather than trusted markup. Chat is not retained after the meeting ends.

Key client operations

Beyond the initial join, a handful of in-meeting actions need careful handling to avoid unnecessary renegotiation or dropped media. The subsections below cover mute and camera toggles, device switching, screen sharing, and quality monitoring.

Mute and camera toggle

Mute and camera toggles should not trigger a full renegotiation in the normal case:

1. Update local intent in `LocalMedia`.
2. Enable or disable the current local track.
3. Send a versioned `update_participant_state` command with a stable command ID, then reconcile the acknowledged room state or show a rejection.

Desired intent, the actual captured/sender track, and server-acknowledged state are separate. Reconnect can safely reapply desired intent after installing the versioned room snapshot, while moderation or authorization failures remain visible instead of silently overwriting it. Camera off is a warm mute in this baseline: disabling the track keeps capture ready for fast resume, and the UI should disclose that the device may remain active.

Device switching

When the user switches microphone or camera devices:

1. Capture a new track from the selected device.
2. Await `RTCRtpSender.replaceTrack()` on the existing sender.
3. Update `LocalMedia` and device preferences.

Only stop the old track after replacement succeeds. On failure, retain the old sender track, stop the new capture, and show a recoverable error; if the new source requires renegotiation, enqueue that work through the same serialized negotiation policy.

✓ **Prefer `replaceTrack()` over renegotiation for device and mute changes**

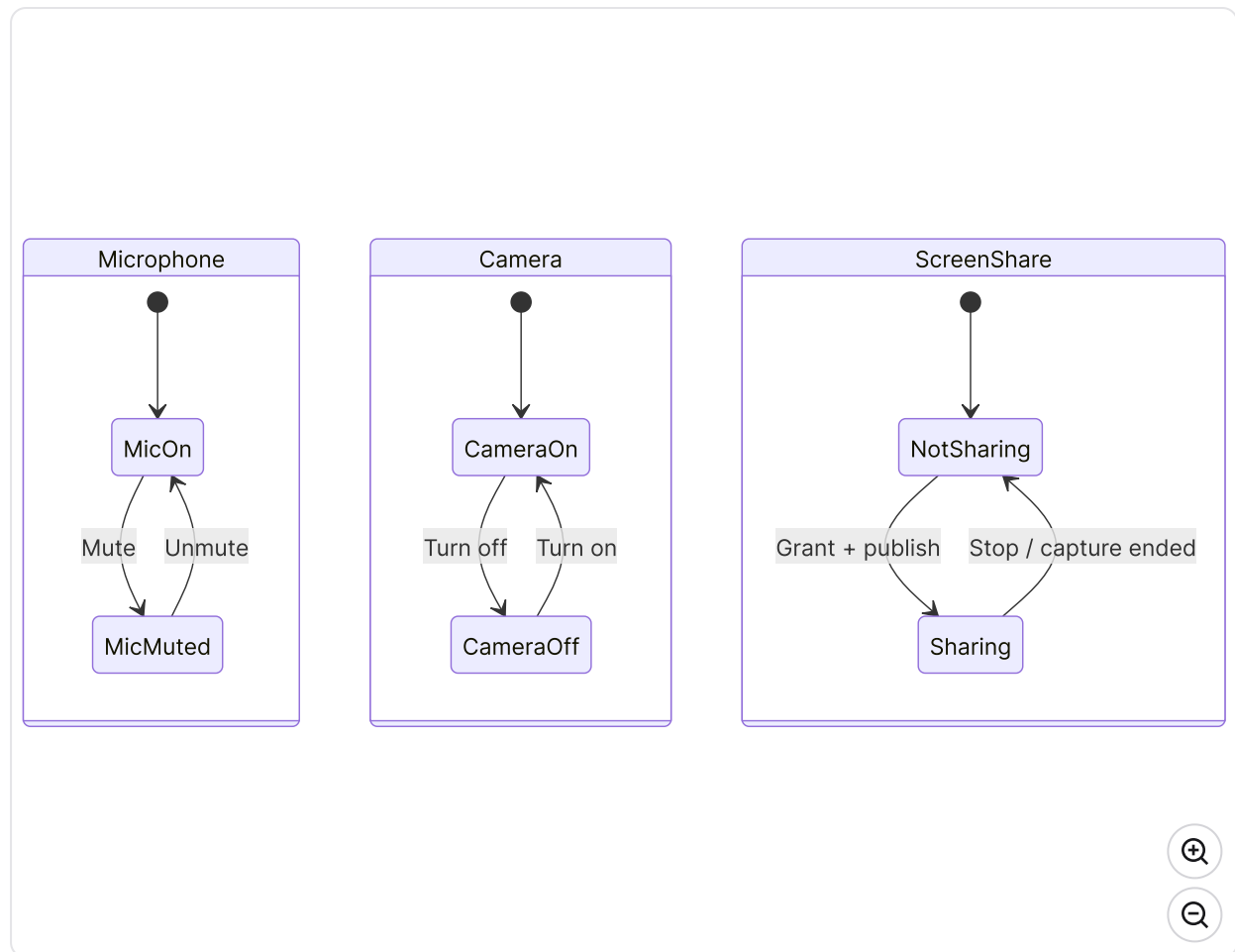
Mute, camera toggles, and device switching should all avoid a full SDP renegotiation in the common case. Toggle `track.enabled` for mute, and call `RTCRtpSender.replaceTrack()` when swapping devices. This keeps the peer connection, SFU routing, and remote tiles stable while the local user changes hardware.

Screen sharing

Screen sharing has a distinct capture lifecycle:

1. The user clicks Share screen.
2. The browser calls `getDisplayMedia()` from that user gesture.
3. The client requests the room's single screen-share grant, then publishes the track and sends the corresponding metadata update.
4. When the browser ends capture or the user stops sharing, the client unpublishes the screen-share track and clears the related state.

For a video conferencing app, it is reasonable to model screen share as a separate published track so that the user can keep the camera and microphone state unchanged. If two participants race, the server grants one request and rejects the other; the losing client stops its captured track and shows that another participant is already sharing.

*Independent local media state transitions*

Joining and leaving are a separate session axis. Starting or stopping screen share changes neither microphone nor camera intent.

Leaving, unmounting, or abandoning a failed join calls one idempotent teardown: stop every owned camera, microphone, and display track; close the peer connection and socket; clear timers and queued work; remove listeners; detach media-element `srcObject` references; and prevent callbacks from an older session generation from updating current state.

Quality monitoring and subscription hints

The client should use `getStats()` to derive a lightweight `StatsSnapshot` for connection-quality UI and adaptive behavior. It does not need to stream raw stats to the server continuously.

Summarize inbound and outbound RTP per publication, plus the selected candidate pair, instead of averaging unrelated tracks. For a simple control loop, sustained packet loss, low inbound bitrate, or decode pressure lowers off-screen and thumbnail layers

first; sustained recovery raises them gradually. Exact thresholds, hysteresis, and device-specific tuning are production follow-ups.

Instead, the client can send occasional `update_subscription_preferences` messages with high-level intent such as:

- which participant is pinned
- whether screen share is visible
- which tiles are on screen

This gives the signaling layer and SFU enough information to prioritize forwarding and quality decisions without turning the API into a stats firehose.

Exercise 1 of 8 Beta

What should happen when a participant joins a meeting after the prejoin lobby?

- 1 Use HTTP for authorized bootstrap, WebSocket for room state and negotiation, and WebRTC for participant media.
- 2 Open signaling and publish muted camera and microphone tracks first, then attach meeting authorization to the established transport.
- 3 Use WebRTC data channels for negotiation, chat, and room commands after HTTP returns credentials and ICE configuration.
- 4 Use HTTP bootstrap for credentials and the initial roster, then poll that endpoint for membership and publication changes during the call.



Reveal answer

Check answer ↩

Optimizations and deep dive

Once the baseline architecture is clear, this section refines it for real-world meeting conditions where bandwidth shifts, device limits, and fast-changing UI state all matter:

- **Adaptive subscriptions and quality selection**: This section explains how the client chooses which remote tracks deserve high quality at any moment.
- **Screen sharing and layout impact**: This section covers how screen share changes layout priority, tile sizing, and media subscription behavior.
- **Reconnect and recovery**: This section discusses network interruptions, session recovery, media renegotiation, and user-visible reconnect states.
- **Rendering performance**: This section explains how to keep video grids, participant state, and frequent UI updates responsive.
- **Browser and UX realities**: This section covers browser permissions, autoplay constraints, device switching, and practical web platform limits.
- **Accessibility and advanced topics**: This section discusses captions, keyboard controls, status announcements, and deeper meeting features.

Adaptive subscriptions and quality selection

The main optimization principle is that not every remote tile deserves the same quality at the same time. A production meeting client should prioritize the following:

- Screen share first, because it is often the primary content people are trying to read.
- Pinned participant or dominant speaker next.
- Visible on-screen participants before off-screen participants.
- Larger tiles ahead of thumbnail tiles.

This is where `LayoutState`, `StatsSnapshot`, and `update_subscription_preferences` start working together. The layout manager already knows which participants matter most to the current view, and the stats layer knows when bandwidth or device capacity is getting tight. Instead of treating every incoming video equally, the client can send higher-level intent to the server and SFU about which tracks deserve the best available quality.

This is also the first line of defense for intermittent or unstable connections. If the call is still alive but packet loss, jitter, or bitrate is getting worse, the client should adapt quality before it drops into a full reconnect flow. In practice, the right behavior is usually to protect intelligibility first and visual richness second.

An unstable connection usually affects the meeting in visible and audible ways before it causes a full disconnect:

- Audio becomes choppy, robotic, or delayed.
- Video tiles become blurry, frozen, or have a lower frame rate.
- Screen share becomes harder to read and less responsive.
- Room-state updates such as active speaker, mute state, roster changes, or chat can feel delayed or briefly out of sync.

That lets us make practical tradeoffs, such as:

- Preserve audio quality before preserving every video tile.
- Keep the main stage or screen share sharp.
- Allow thumbnails to drop resolution first.
- Reduce quality for participants who are off screen.
- Reduce incoming video load before abandoning the session entirely.
- Recover quality as layout importance or network conditions improve.

✓ **Protect audio intelligibility before visual richness under bandwidth pressure**

Conversations survive blurry tiles and frozen thumbnails far better than they survive choppy or robotic audio. When packet loss or bitrate degrades, drop thumbnail resolution and off-screen subscriptions first, keep the pinned speaker or screen share sharp, and only downgrade audio as a last resort before giving up the session.

For a more scalable production design, the natural extension is simulcast or layered encodings. In WebRTC, simulcast means publishing multiple simultaneous encodings of the same source at different resolutions or bitrates so that the SFU can forward the best-fit version to each subscriber:

- The publisher sends multiple quality layers for the same camera track.
- The SFU forwards the best-fit layer for each subscriber.
- The client can react to layout changes without forcing a full renegotiation every time the active speaker changes.

In an interview, it is enough to say that the baseline should support per-tile prioritization and that simulcast is the production technique that makes this efficient at

larger room sizes.

Exercise Beta

Available bandwidth falls during a 25-person call with a pinned screen share and several off-screen participants. Which subscription changes preserve the most useful meeting experience? Select all that apply.

- ☐ 1 Give screen share, pinned participants, dominant speakers, and large visible tiles higher quality.
- ☐ 2 Assign every visible track the same quality layer and rely only on transport congestion control to share the bandwidth.
- ☐ 3 Lower layers or unsubscribe off-screen tiles and recover quality when they become important.
- ☐ 4 Preserve intelligible audio before high-resolution thumbnail video.



Reveal answer

Check answer ↩

Screen sharing and layout impact

When a participant starts sharing their screen, the meeting layout shifts fundamentally: the screen-share track becomes the dominant tile, and camera feeds collapse into a secondary strip or sidebar. This transition directly affects adaptive subscription logic because the screen-share track now deserves the highest quality budget, while camera thumbnails can drop to lower simulcast layers.

Screen content also has different encoding characteristics than webcam video. Slides, code, and documents benefit from higher resolution at lower frame rates to keep text sharp, while a demo showing motion content needs the opposite tradeoff. The `contentHint` property on `MediaStreamTrack` lets the client signal this preference to the browser encoder: setting it to `"text"` or `"detail"` prioritizes sharpness, while `"motion"` prioritizes smoother frame delivery. Applying the right hint at capture time helps the encoder make better decisions without requiring additional server-side logic.

The layout manager should also handle the end of screen sharing cleanly. When the browser ends capture asynchronously or the presenter stops sharing, the layout needs to revert to the previous arrangement, and the subscription priorities need to readjust so that camera tiles regain their normal quality allocation.

Exercise Beta

A presenter alternates between a static code editor and a motion-heavy prototype demo. Which screen-share encoding policy best matches the content?

- 1 Apply the presenter's current camera encoding preference to the display track so both publications adapt together.
- 2 Always prioritize maximum spatial detail because every display track is more readable at the highest available resolution.
- 3 Favor spatial detail at a lower frame rate for code, then favor smoother frame delivery for the motion-heavy demo.
- 4 Always prioritize the highest frame rate because pointer movement is present in both the code editor and the prototype demo.

[Reveal answer](#)[Check answer ↩](#)

Reconnect and recovery

Reconnect and recovery matter more in video conferencing than in many other products because the session is long-lived, interactive, and extremely sensitive to interruption. Users move between networks, laptops sleep and wake, VPNs or captive portals interfere with connectivity, and browser tabs can briefly lose their real-time connection even while the meeting is still in progress.

In a news feed, a retry usually just means waiting a little longer for content. In a video conferencing product, a failed recovery path can mean missing part of a conversation, talking over others because audio has frozen, or thinking the room is empty when the roster is merely out of sync. That is why reconnect behavior is part of the core architecture, not just an edge-case optimization.

The handoff from the previous subsection is important: adaptation is for unstable connections that are still alive, while reconnect and recovery are for sessions that are broken, flapping, or no longer trustworthy.

Reconnect logic is easier to reason about if we separate signaling recovery from media-path recovery instead of treating everything as one generic failure mode.

★ **Model signaling recovery and media-path recovery as two separate flows**

Signaling can fail while media is still flowing, and media can degrade while the WebSocket looks healthy. Treating the whole thing as one "connection" causes UI flicker, unnecessary full reconnects, and room-state resyncs that tear down otherwise working peer connections. Drive signaling recovery through fresh `room_snapshot` fetches and media-path recovery through ICE restart before escalating to a full session rebuild.

- Signaling recovery is about restoring the WebSocket session and resynchronizing room state.
- Media-path recovery is about restoring a healthy WebRTC path when ICE connectivity degrades or fails.

For signaling recovery, the conference controller should:

- Reopen the WebSocket connection.
- Request a fresh `room_snapshot` instead of trying to replay every missed delta.
- Reapply local participant intent such as muted state, camera-off state, or subscription preferences after installing the versioned snapshot.

While an active WebSocket connection is ordered and reliable, clients can still miss signaling events across disconnects, sleep/wake cycles, or reconnect boundaries. That is exactly why resynchronizing from a fresh snapshot is safer than assuming that the local room state is still complete.

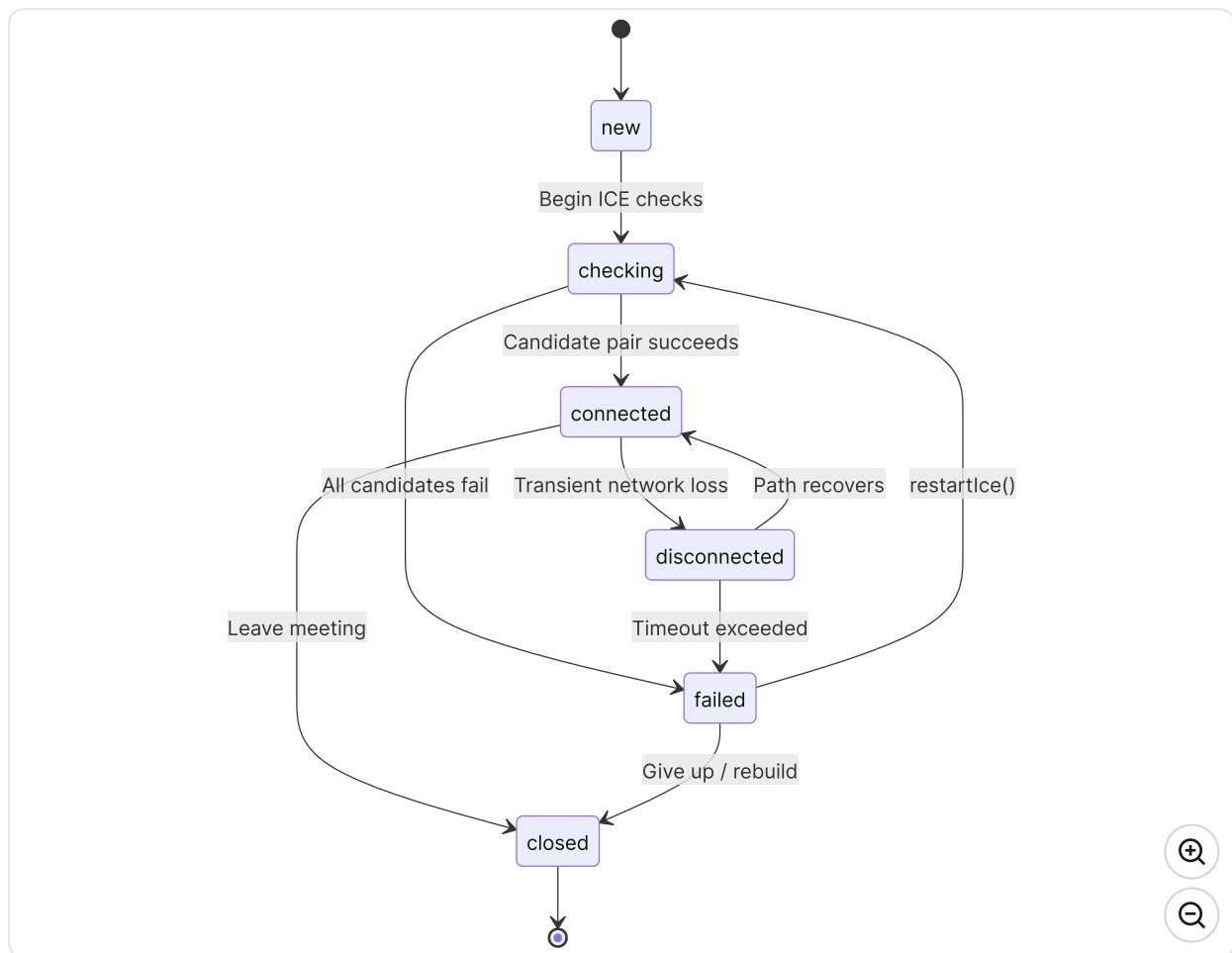
For media-path recovery, the peer connection manager should:

- Detect degraded or failed ICE state through connection-state callbacks.
- Attempt ICE restart through the serialized negotiation queue before escalating to a full session rebuild. An ICE restart gathers fresh network candidates and

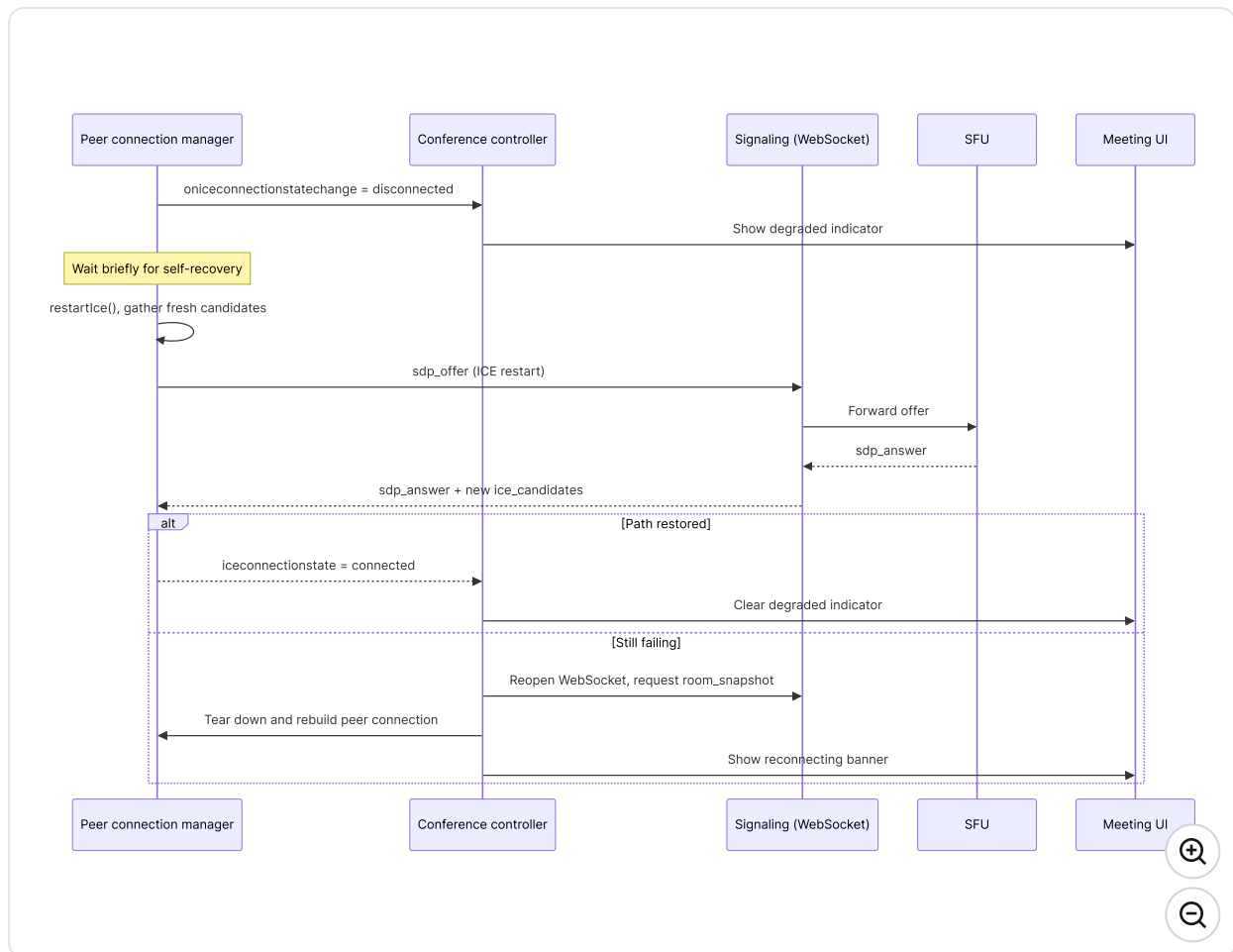
renegotiates the transport path without tearing down the entire peer connection, which makes it much cheaper than a full reconnect.

- Ensure configured TURN servers are available so ICE can retry with relay candidates when direct paths are no longer viable.

This split matters because signaling can fail while media is still briefly alive, and media can degrade even when signaling is healthy. Modeling them separately helps avoid UI flicker, reduces unnecessary full reconnects, and makes retry behavior more predictable.



ICE connection state transitions driving recovery



Reconnect after a brief network blip via ICE restart

The client should also surface recovery state clearly:

- Show a reconnecting banner when signaling is down.
- Show a degraded connection indicator when media quality drops.
- Show an actionable error state if recovery fails after repeated attempts.

That keeps the system understandable from the user's point of view. Even when the client cannot fully hide transient network issues, it can still preserve trust by making the current state clear.

Exercise Beta

Signaling disconnects while audio continues, then one participant's media path fails after the socket recovers. Which policies recover each layer without unnecessary teardown? Select all that apply.

1

Pause all working media and rebuild peer connections before installing the recovered signaling snapshot.

☐

2

Keep TURN relay configuration available when direct candidate paths no longer work.

☐

3

Attempt an ICE restart for a failed media path before rebuilding the whole peer connection.

☐

4

Reconnect signaling, buffer newer events, install a generation- and sequence-bounded room snapshot, then drain only contiguous deltas.

☐

5

Once signaling reconnects, mark media recovered unless a peer connection has already entered `failed` state.

☐



[Reveal answer](#)

[Check answer ↩](#)

Rendering performance

Rendering performance is unusually important in a video conferencing app because the UI is driven by many fast-changing signals at once:

- Audio and video tracks are starting, stopping, and switching devices.
- Mute state, active speaker, and roster state can change many times during a short meeting.
- The layout can change when participants join, leave, pin someone, or begin screen sharing.
- Stats and connection quality indicators can update continuously in the background.

If we couple all of that too directly to normal React render paths, the meeting UI can become janky at exactly the moment when users most need it to feel stable. The main goal is to keep the media lifecycle more stable than the surrounding UI lifecycle.

A few practical rules follow from this:

- Keep participant tiles mounted when possible instead of remounting them on every layout change.
- Attach tracks to existing media elements through refs and `srcObject` instead of treating them like ordinary serializable props.

- Keep `RTCPeerConnection`, senders, tracks, and stats polling outside hot render paths.
- Collapse noisy transport and stats updates into a smaller UI-facing view model before they trigger rerenders.

This is especially important because the cost is not just visual polish. Poor rendering strategy can cause:

- Delayed mute or speaking indicators.
- Stuttering local preview.
- Flickering tiles when dominant speaker changes.
- Unnecessary media reattachment when only metadata changed.
- Sluggish controls during participant churn.

A few implementation techniques help keep the experience responsive:

- Batch frequent participant and quality updates before committing them to UI state.
- Avoid rerendering the whole participant grid when only one participant's badge or quality indicator has changed.
- Separate local preview concerns from remote tile concerns so device changes stay localized.
- Treat side panels such as chat or roster as independent surfaces so that they do not force the video grid to rerender unnecessarily.

This is similar in spirit to the [Netflix video streaming system design article](#): media elements need a more stable lifecycle than the rest of the page around them.

Exercise Beta

A 25-person meeting grid flickers when the dominant speaker changes and rerenders every second with new transport statistics. Which rendering changes address those symptoms? Select all that apply.

1

Store each peer connection in its participant tile so media and UI share one lifecycle.

☐

2

Keep participant media elements mounted across layout changes and attach tracks imperatively through refs and srcObject.

☐

3

Isolate chat, roster, local preview, and video-grid render paths.

☐

4

Reduce noisy stats and transport events to bounded UI state and update only affected participant tiles.

☐

Reveal answer

Check answer ↩

Browser and UX realities

Video conferencing on the web is constrained not just by our app architecture but also by browser security rules and platform behavior. If we ignore those constraints, the product can feel flaky even when the network and media stack are otherwise healthy.

The most important browser realities are:

- Camera, microphone, and display-capture APIs require secure contexts, so the meeting experience must be served over HTTPS.
- Permissions can be denied, dismissed, delayed, or revoked after the meeting has already started.
- Remote audio playback can be affected by autoplay rules.
- Device availability can change while the meeting is in progress.
- Screen sharing has its own capture lifecycle that is partly controlled by the browser, not just by the app.
- Speaker output selection is not uniformly supported, so some browsers may need to fall back to the system default output device.



Capture APIs require HTTPS and a fresh user gesture for screen share

`getUserMedia()` and `getDisplayMedia()` only work in secure contexts, so staging or preview environments served over plain HTTP will fail before the meeting even starts. `getDisplayMedia()` also cannot reuse a prior permission grant, meaning screen share must always be triggered from a direct user action and the client must listen for asynchronous capture-ended events to unpublish cleanly.

These constraints create product requirements, not just implementation details:

- The production deployment needs HTTPS for capture APIs to work at all.
- The prejoin lobby should model requesting, pending, denied, and unavailable states instead of collapsing them into a generic join error. A pending request can be canceled, and users can join with the affected media off; navigation or retry invalidates the capture attempt so a late result is stopped rather than attached. Denied states link to browser-specific instructions for reopening permissions.
- The in-meeting UI should handle blocked playback gracefully instead of assuming remote audio will always start automatically.
- Device pickers should refresh and fall back safely when microphones, cameras, or speakers disappear.
- Speaker pickers should be capability-gated and fall back to the default output device when explicit output selection is unavailable.
- Screen-share state should clean itself up immediately when browser capture ends outside the app's control.

A few concrete browser behaviors are worth naming:

- The `devicechange` event can fire when audio or video devices are added or removed, which helps keep the device state fresh during long meetings.
- `getDisplayMedia()` must be triggered from a user action, which is why screen sharing belongs behind an explicit Share screen control.
- `getDisplayMedia()` permission cannot be silently reused, so browsers prompt the user each time they start a new screen share.
- Audio output selection depends on APIs such as `selectAudioOutput()` and `setSinkId()`, which are capability-gated and not uniformly available.
- Screen-share tracks can end asynchronously, so the client needs to listen for that event and unpublish the share promptly.
- Browser-specific behavior is not perfectly uniform, so capability detection and graceful degradation matter more than assuming every feature works identically everywhere.

If time permits, this is also a reasonable place to mention browser-native tuning such as `contentHint` for prioritizing text clarity during screen sharing versus smoother motion for webcam video.

Exercise 1 of 2 Beta

A returning user has previously granted camera access and shared a screen, but now joins with a disconnected microphone. Which browser constraints still require explicit UX handling? Select all that apply.

- ☐ 1 Treat the previous screen-share grant like camera permission and offer an automatic resume toggle on the next call.
- ☐ 2 Serve capture flows in a secure context and model pending, denied, dismissed, revoked, and unavailable camera or microphone access.
- ☐ 3 Call `getDisplayMedia` from a fresh user gesture and listen for the browser ending the track.
- ☐ 4 Handle `devicechange`, fall back when hardware disappears, and capability-gate speaker output selection.



Reveal answer

Check answer ↩

Accessibility and advanced topics

Accessibility matters a lot in video conferencing because the product has dense controls, rapidly changing status indicators, and long sessions where users need to react quickly without losing context. A meeting UI that is technically functional but hard to navigate with a keyboard or screen reader becomes exhausting very quickly.

At a minimum, the product should treat these as baseline expectations:

- Mic, camera, and screen-share toggles expose pressed state, clear labels, and visible shortcuts within a keyboard-navigable toolbar.
- Focus stays on the invoking control when panels or layouts change, and shortcuts do not fire while the user is typing in chat.
- Participant tiles use stable accessible names and reading order; muted or speaking state does not rely on color alone.
- Chat uses log semantics with rate-limited announcements so a burst of messages does not overwhelm other meeting updates.

- The layout reflows at browser zoom without hiding the leave control or trapping keyboard users in a panel.
- Important states such as reconnecting, blocked permissions, or ended screen share use concise ARIA live announcements without moving focus.

This is also where we should draw the line between the baseline architecture and advanced product work. There are many useful extensions, but they should not distract from the core meeting design:

- Live captions and transcription.
- Reactions and hand raise.
- Breakout rooms.
- Advanced audio features such as same-room adaptive audio.
- Background blur and heavier media effects.

These are all reasonable follow-up topics, but the baseline answer should focus first on dependable join flow, intelligible audio, stable video, and understandable room state.

Other potential follow-ups are perfect negotiation for two-sided offers, signaling credential renewal and revocation at scale, a privacy control that fully releases and later reacquires camera hardware, application-level media E2EE, and production tuning for quality thresholds and cooldowns. Captions remain out of scope for the baseline, which limits accessibility for participants who cannot rely on meeting audio.

Exercise Beta

The active speaker changes frequently while a keyboard user is typing in meeting chat. Which behaviors keep controls operable and expose meeting changes without disrupting the user's current task? Select all that apply.

1 Move focus to the active speaker tile only when the grid also promotes that tile to the main stage. ☐

2 Use a labeled chat log, rate-limit routine announcements, and announce important failures without stealing focus. ☐

3 Keep focus in chat but announce every active-speaker change through an assertive live region. ☐

4

Give mute, camera, and share semantic toggle state, and keep all core controls stably labeled and keyboard accessible.



Reveal answer

Check answer ↩

Summary

A Zoom-style conferencing design scoped to roughly 25 active participants resolves into a chain of dependent decisions. Present that chain top-down rather than listing every piece at once.

Choose the SFU topology because every later choice inherits from topology. A full-mesh design collapses once each client has to encode and upload a separate stream to every peer, and an MCU server-side composite saves bandwidth but hard-codes layout, active-speaker logic, and per-tile quality into a pre-rendered frame. An SFU lets each client publish a bounded set of tracks and encodings over one upstream peer connection, lets the server forward selectively to each receiver, and keeps layout and per-tile behavior owned by the UI.

Split backend communication across HTTP bootstrap, WebSocket signaling, and WebRTC media. A short-lived HTTP bootstrap fetches meeting metadata and ICE server configuration for the prejoin screen, a persistent WebSocket signaling channel carries room state, browser offers, SFU answers, ICE candidates, and chat, and a WebRTC media path carries audio, video, and screen share to and from the SFU while STUN and TURN let ICE find a working path across NATs and restrictive networks.

Decompose the client so each protocol has a clear owner. An app shell owns routing and auth, a prejoin lobby gates the session on device permissions and self-preview, a conference controller orchestrates the in-meeting lifecycle, a peer connection manager wraps the WebRTC surface, a device manager encapsulates `getUserMedia` and `getDisplayMedia`, a layout manager resolves speaker view and gallery view, and a chat and presence panel consumes the same signaling feed that drives the roster.

Separate serializable meeting state from live browser handles. Server-originated entities such as `Meeting`, `Participant`, `TrackPublication`, `ChatMessage`, and

`ScreenShareSession` belong in a normalized store keyed by ID so roster, tiles, and chat stay consistent under churn. Client-only values such as `LocalMedia`, `Permissions`, `ScreenShareState`, `ConnectionState`, `LayoutState`, and `StatsSnapshot` stay in memory. Live objects such as `MediaStream`, `MediaStreamTrack`, and `RTCPeerConnection` stay imperative, owned by the peer connection manager, and are attached to `<video>` elements through refs and `srcObject`.

Protect perceptual quality and recovery at the right layer. Adaptive subscriptions and simulcast let the client pull the right layer for each tile, audio-first prioritization protects intelligibility before visual richness when bandwidth tightens, and signaling recovery stays distinct from media recovery: a dropped WebSocket reconciles through a versioned `room_snapshot`, while a stalled media path queues ICE restart before a full peer rebuild. Mute and camera toggles use `track.enabled`; device switches await `RTCRtpSender.replaceTrack()` and fall back to serialized renegotiation when necessary.

In an interview, start with the SFU choice, the three-protocol split, and the boundary between normalized meeting state and imperative media handles. Everything else in the design follows from those three.

References

The references below group official browser and WebRTC API documentation alongside related system design articles on this site.

Official WebRTC and browser documentation

These references cover the WebRTC and media-device APIs that the designs in this article build on directly.

- [WebRTC API - MDN](#) for the overall browser media and connection model.
- [RTCPeerConnection - MDN](#) for negotiation, connection state, and track transport.
- [Perfect negotiation pattern - MDN](#) for collision-safe negotiation handling.
- [MediaDevices.getUserMedia\(\) - MDN](#) for prejoin camera and microphone capture.

- [MediaDevices.getDisplayMedia\(\) - MDN](#) for screen-sharing behavior and constraints.
- [MediaDevices.selectAudioOutput\(\) - MDN](#) for browser-gated speaker selection.
- [HTMLMediaElement.setSinkId\(\) - MDN](#) for routing audio to a selected output device where supported.
- [RTCRtpSender.replaceTrack\(\) - MDN](#) for device switching and track replacement.
- [RTCPeerConnection.restartIce\(\) - MDN](#) for media-path recovery after connectivity failure.
- [RTCPeerConnection.getStats\(\) - MDN](#) for connection-quality monitoring and adaptive behavior.
- [MediaStreamTrack.contentHint - MDN](#) for screen-share and webcam encoding hints.

Related system design articles

The articles below overlap with video conferencing on real-time transport, reconnect behavior, and media element handling.

- [Chat Application \(e.g. Messenger\)](#) for real-time events, presence, and reconnect-oriented UI behavior.
- [Video Streaming \(e.g. Netflix\)](#) for stable media-element lifecycles and quality-adaptation practices.

Exercises

Exercise 1 of 22 Beta

A team is defining a Zoom-like browser meeting client before selecting media topology and protocols. Which scope gives that decision a useful foundation?

- 1 Scope a one-to-one call first, assume peer-to-peer WebRTC, and leave group-size and recovery constraints to implementation.
- 2 Define codecs, TURN deployment, and signaling messages in detail, using current Zoom behavior as the implicit product contract.

3

Treat scheduling, live media, recording, webinars, captions, breakout rooms, reactions, and virtual backgrounds as equal baseline concerns.

4

Bound participant scale, active media, target browsers, essential meeting actions, quality priorities, reconnect behavior, accessibility, and excluded follow-ups.



Reveal answer

Check answer ↩

Discussions



Normal T

B

I

U



<>

Aa



</>



Type comment here...

0/1000

Post



Report an issue



✓ Mark complete



Subscribe to our newsletter

Notify me

[Practice](#)[Study Plans](#)

Get started

JavaScript functions

User interface coding

System design

Quiz

Guides

Front End Interview Playbook

Front End System Design Playbook

React Interview Playbook

Behavioral Interview Playbook

1 Week Plan

1 Month Plan

3 Months Plan

Company

Pricing

Promotions

Roadmap

About

Team

Contact us

Become an affiliate

Careers

Blog

Medium

DEV Community



English (US) ▾